



Natural Language Processing

Tweet Sentiment Analysis

Nov 2025

Maria Schoinaki

Electra Papadopoulou

Leonidas Kontogiannis

You can view the full code implementation in the following link:



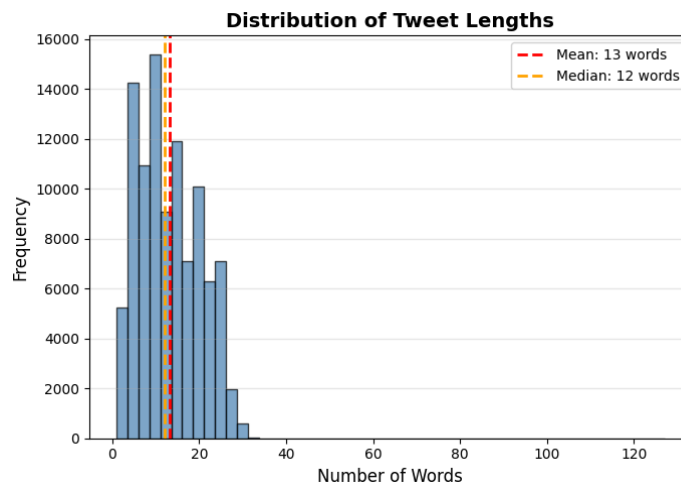
For this assignment we used the [Twitter Sentiment Analysis](#) dataset from Kaggle. This dataset consists of ~100.000 tweets (sentiment texts), the id of each tweet, and the sentiment (0 or 1), and it was used as the basis for training (70%), development (15%), and testing (15%) of the sentiment classifiers.

All members have contributed equally in all tasks, markdown and report writing, synchronously throughout the assignment.

Data Exploration

We created a bar chart to display the distribution of sentiment labels (0 for negative and 1 for positive), via which we can verify whether the dataset is balanced or biased towards a specific label. The resulting chart shows a total of **99,989 tweets**, of which **43.5%** are negative, and the other **56.5%** are positive. There is a slight predominance of positive tweets, but it is not severe enough to consider when training the classification models.

Then, we plot a histogram of tweet lengths that we measure by the number of words per tweet.



By these values we can tell that the majority of tweets (as expected) are concise, around 12-13 words, though there exists a small number of longer tweets.

Data Preprocessing/ Cleaning

In order to preprocess the dataset, we implement **spaCy** and **regular expressions**.

- **Text Cleaning:**

For a dataset full of tweets, it is important to scrap the information that is not useful for sentiment classification, such as **symbol-prefixed words**; Twitter utilizes special characters for mentions (*@username*) or tagging (*#hashtag*) and at times quoting (*"e*), so we make sure to eliminate those tokens from our dataset. Additionally, we remove all **HTML entities** (encoded symbols such as *"e;* or *&*), and **URLs** (*http://*, *https://* or *www.*). None of these contribute valuable semantic information for sentiment analysis. Non-word characters and extra spaces are also removed to standardize formatting. Lowercasing is applied to ensure there's case consistency across tokens.

- **Linguistic Normalization:**

To preprocess each tweet, we will utilize **Part-Of-Speech (POS)** tagging. It's a process of assigning grammatical categories (roles) to words based on their context, which allows for identifying sentence structure and improves lemmatization by knowing the word's type. The same word can have multiple tags. We opted to do this using spaCy's **English language model** (*en_core_web_sm*), which uses deep learning (CNN/transformer) and considers context, but it takes some time to train. The model's parser and named-entity recognizer were disabled to improve efficiency, as only **tokenization** and **lemmatization** were required.

The result of the process is a list of tweets. Here are some examples of the results:

Original tweet text:

@ashers11 Why?

Filtered tweet text:

why

Original tweet text:

@akiraLOVE haha, yeah hey, he replied to you too right??

Filtered tweet text:

haha yeah hey he reply to you too right

Original tweet text:

wakin up exactly where i wanna be...

Filtered tweet text:

wakin up exactly where wanna be

An issue that is prevalent in the dataset is that a lot of words are misspelled or with additional/missing letters, ('why' instead of 'why', 'wakin' instead of 'waking') and thus they are not preprocessed correctly, but it would be very difficult to find a method that corrects all of those mistakes - tweets are known for their use of slang words or exaggerations, and it would be a new problem entirely to completely normalize them.

Experimental Setup

- **Corpus:** twitter-sentiment-analysis2 from Kaggle.
- **Splits:** 70% *train*, 15% *dev*, 15% *test* (*random_state* = 2025).
- **Feature Extraction:** TF-IDF (unigram, bigram and trigram, *max_features*=30000, *sublinear_tf*= *True*).
- **Dimensionality Reduction:** Truncated SVD (*n_components*=2, *random_state*=2025)
- **Classifiers:** Baseline, TF-IDF Logistic Regression, Boolean Logistic Regression, kNN

Feature Extraction (TF-IDF n-gram features)

We extract the text features by using **TfidfVectorizer** and configuring it with unigram, bigram and trigram features, and a vocabulary limit of 30,000 features. Then we fit it on the training data and apply it to both training and test sets and we extract the corresponding **TF-IDF vector** from the individual test document. Each feature (word or n-gram) is paired for its TF-IDF weight for a

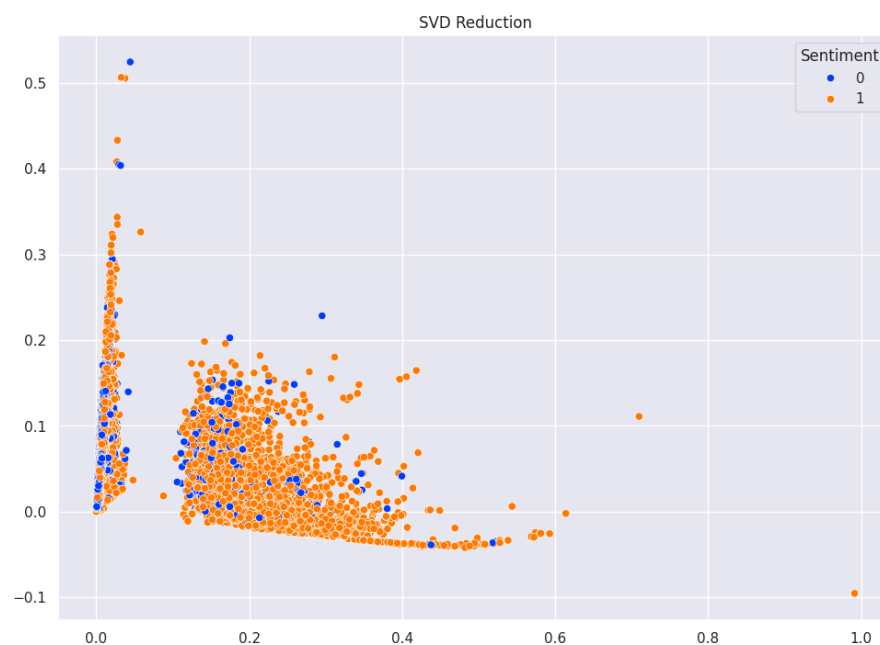
feature-weight map, and here we show the non-zero entries, the words actually present in the tweet. We get a list of the most influential words, as well as their weights.

```
Number of non-zero features in this document: 4
      Feature  TFIDF
24663  spill  0.5822
8928   homie 0.5616
4850    dm   0.4338
26963  true  0.3968
```

Most entries are zero, because each tweet contains only a small subset of all possible words. Only a small fraction of the 30,000 features are non-zero per tweet.

Dimensionality Reduction (For Visualisation)

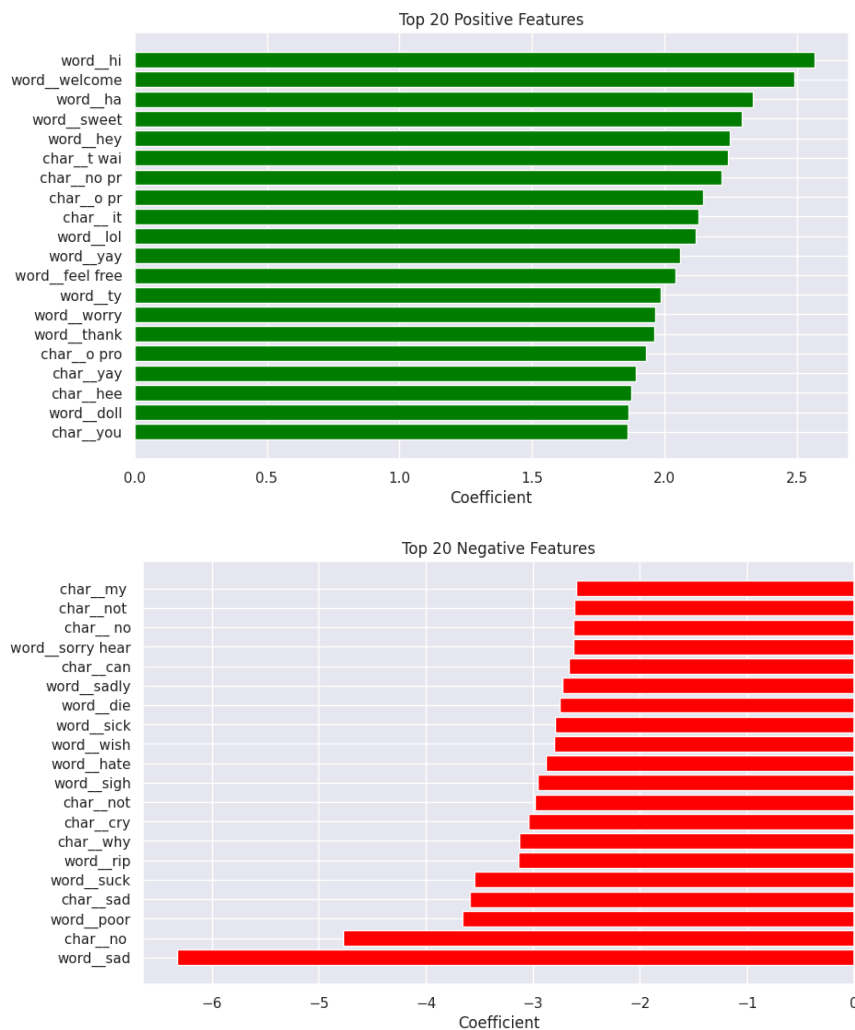
In order to visualise this feature space in two dimensions, we apply **TruncatedSVD** to reduce the data in two components, so we can be informed of the variance in our data and the general structure of the feature space.



Why not use PCA? Because Truncated SVD works directly with sparse matrices so it's good for TF-IDF representations. We see here that there is an overlap between the two classes, as the orange points representing the positively labeled tweets are overlapping with the blue ones that represent the negative tweets. This makes it more difficult for a linear model to perform well.

Feature Selection With Information Gain (Top Features)

After the feature extraction, we compute the information gain for each candidate feature and select the (k=30000) best features. We inspected the **top info-gain features** (*i.e.*, the words and character n-grams with the strongest positive and negative coefficients). **Positive coefficients** indicate features that **push** the **prediction** toward the **Positive class**, while **negative coefficients** indicate features associated with the **Negative class**. The feature extraction pipeline included **both word-level n-grams and character-level n-grams**, which allows the model to capture both standard vocabulary and informal Twitter spelling patterns.



Overall, the feature inspection **confirms** that the model is **not simply memorizing** isolated keywords but is **capturing emotionally meaningful patterns**, including **informal language**, **negation**, **sympathy expressions**, and **conversational tone**, all highly characteristic of social-media sentiment.

Linear Classification with Logistic Regression

For the classification stage, we employ **Logistic Regression**, a linear model that performs well on high-dimensional, sparse data such as **TF-IDF** text representations. Logistic Regression estimates the probability that a given input belongs to a particular class based on a linear combination of the feature weights learned during training. In this context, each feature corresponds to an n-gram (*word or character*), and its learned coefficient reflects how strongly it influences sentiment polarity.

We train two main Logistic Regression variants for comparison:

- **TF-IDF Logistic Regression:** Uses term frequency–inverse document frequency weighting to capture both the importance of individual words within a tweet and their relative rarity across the corpus. This model benefits from capturing both unigram and higher-order n-grams (*bigrams and trigrams*), allowing it to learn short multi-word expressions such as “not good” or “very happy”.
- **Boolean Logistic Regression:** Uses binary term presence (*1 if the token appears, 0 otherwise*) rather than TF-IDF weighting. This version focuses on whether a feature occurs at all, disregarding frequency. While this can reduce the influence of very frequent terms, it may lose some nuance in weighting important repeated words.

Both models are trained with **L2 regularization** to prevent overfitting. The regularization strength is controlled by the parameter **C** (inverse of regularization coefficient λ). A higher C value allows the model to fit more closely to the training data, while a smaller C enforces stronger regularization, promoting generalization. Through HyperParameter Tuning the best value was found to be **C = 1**, with a feature selection size of **k = 30,000** using **SelectKBest** based on mutual information scores.

Non-Linear Classification with K-Nearest Neighbors (KNN)

Besides the Linear Classification baselines that we created previously, we went for a different approach by implementing a **k-nn** classifier, using the **TF-IDF** representation of the data (*vectorizer*). This method takes a number of training samples k (in this case k=25) to compare a new observation to, and the class label is predicted by **majority vote**. The votes are also weighted by similarity, making it so that closer neighbors count more to the classification. K represents the closest tweets to the observation according to a similarity metric, usually Euclidean distance. However, for text vectors, **cosine** distance is preferred, as it measures the angle between vectors, focusing on the relative composition of words. We add the model to the previous set of classifiers, and evaluate it on the same metrics as we did before.

HyperParameter Tuning

To improve the performance of the **TF-IDF + Logistic Regression** classifier, we performed hyperparameter tuning using **GridSearchCV**. Since the assignment requires that hyperparameters be tuned on the **development (dev) set**, we used **PredefinedSplit** to enforce a **fixed train–dev** split within **GridSearchCV** rather than using **cross-validation** folds. This ensures that model selection does not accidentally leak information from the test set or reassign data partitions.

The tuning results show that:

- Including **trigrams** and **larger vocabulary sizes** improves sentiment classification in **Twitter** data, where **expressive phrasing** and **informal constructions** are **common**.
- **Character-level** features significantly **enhance** robustness to **spelling variation**, **emoji-like expressions**, and **slang**.
- Choosing $C = 1$ ($C=1/\lambda$) **balances** model **generalization** and **avoids overfitting** observed at larger C values.
- Retaining a **high number of informative features** ($k = 30,000$) helps **maintain discriminative power in short texts**.

Overall, the tuned model is more stable and generalizes better to unseen tweets.

Evaluation

We report per-class **Precision (P)**, **Recall (R)**, **F1**, and **PR-AUC** (*area under the Precision-Recall curve, computed one-vs-rest from predict_proba*). We also include **Macro-Averaged P/R/F1/PR-AUC** (*mean over classes*). Models are **trained** on **train** dataset, **tuned** in **dev** dataset and **evaluated** in **test** dataset. No test data are used for tuning. Vectorizers and any selectors are fit only on the training set via scikit-learn pipelines.

Systems Compared

- **Majority baseline** (*dummy classifier-most_frequent*): Always predicts the training set's majority class.
- **LR (Boolean)**: Logistic Regression with binary Bag-of-Words (*presence, absence*).
- **LR (TF-IDF) tuned**: Logistic Regression with TF-IDF word n-grams (*uni, bi, tri*).
- **KNN (TF-IDF)**: K-nearest neighbors with TF-IDF word n-grams (*uni, bi, tri*).

We first trained a **simpler Logistic Regression** model using **Boolean (binary) bag-of-words** features **without hyperparameter tuning**. This model serves as a **strong classical baseline**, complementing the majority-class baseline. We then constructed a **TF-IDF based Logistic**

Regression pipeline and performed **hyperparameter tuning** using the development set. The **tuned TF-IDF model** consistently **outperformed the Boolean baseline**, demonstrating that both richer feature representations and appropriate hyperparameter optimization substantially improve classification performance.

Results

We provide three tables (*train, dev, test*). Each table has one row per model and four columns per class (*P, R, F1, PR-AUC*), followed by *Macro-P, Macro-R, Macro-F1, Macro-PR-AUC*.

Training set

=== TRAIN RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Majority Baseline	0.0000	0.0000	0.0000	0.4354	0.5646	1.0000	0.7217	0.5646	0.2823	0.5000	0.3609	0.5000
1	TF-IDF+LR	0.7940	0.8281	0.8107	0.8793	0.8629	0.8344	0.8484	0.9305	0.8285	0.8312	0.8296	0.9049
2	Boolean+LR	0.7912	0.8010	0.7960	0.8632	0.8451	0.8370	0.8410	0.9160	0.8181	0.8190	0.8185	0.8896
3	TF-IDF+kNN	0.9898	0.9976	0.9937	0.9998	0.9982	0.9921	0.9951	0.9998	0.9940	0.9948	0.9944	0.9998

- The **majority baseline** reaches Macro-F1 ≈ 0.36 and Macro PR-AUC ≈ 0.50 (*constant prediction yields poor F1 but PR-AUC near the class prior*).
- **LR** models achieve high training scores. **TF-IDF** slightly outperforms **Boolean**, indicating that term-frequency information and n-grams help.
- **k-NN** achieves **extremely high training accuracy** (Macro-F1 ≈ 0.995), indicating **severe overfitting**: nearest-neighbor methods **memorize high-dimensional sparse vectors** and do **not generalize** well **without additional regularization** or **dimensionality reduction**.

Development set

=== DEV RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Majority Baseline	0.0000	0.0000	0.0000	0.4354	0.5646	1.0000	0.7217	0.5646	0.2823	0.5000	0.3609	0.5000
1	TF-IDF+LR	0.7220	0.7585	0.7398	0.8055	0.8062	0.7748	0.7902	0.8763	0.7641	0.7666	0.7650	0.8409
2	Boolean+LR	0.6851	0.6888	0.6870	0.7560	0.7590	0.7559	0.7575	0.8312	0.7221	0.7224	0.7222	0.7936
3	TF-IDF+kNN	0.7002	0.5977	0.6449	0.7279	0.7212	0.8027	0.7598	0.8188	0.7107	0.7002	0.7023	0.7734

- **Baseline:** Macro F1 ≈ 0.3609 , Macro PR-AUC=0.5000
- **LR (Boolean):** P=0.7221, R=0.7224, F1=0.7222, PR-AUC=0.7936
- **LR (TF-IDF):** P=0.7641, R=0.7666, F1=0.7650, PR-AUC=0.8409
- **KNN (TF-IDF):** Macro F1=0.7023, Macro PR-AUC=0.7734

Observation. TF-IDF LR clearly dominates Boolean LR AND kNN on Dev ($\approx +4.3$ Macro-F1, $+0.047$ PR-AUC). The baseline is far behind, as expected.

Test set

=== TEST RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Majority Baseline	0.0000	0.0000	0.0000	0.4354	0.5646	1.0000	0.7217	0.5646	0.2823	0.5000	0.3609	0.5000
1	TF-IDF+LR	0.7282	0.7508	0.7394	0.8103	0.8032	0.7839	0.7934	0.8760	0.7657	0.7674	0.7664	0.8431
2	Boolean+LR	0.6895	0.6897	0.6896	0.7551	0.7607	0.7605	0.7606	0.8255	0.7251	0.7251	0.7251	0.7903
3	TF-IDF+kNN	0.7073	0.6025	0.6507	0.7371	0.7249	0.8078	0.7641	0.8206	0.7161	0.7051	0.7074	0.7789

The **baseline** predicts only the majority class → **zero** performance for the minority class and **flat PR-AUC ~ class prior**.

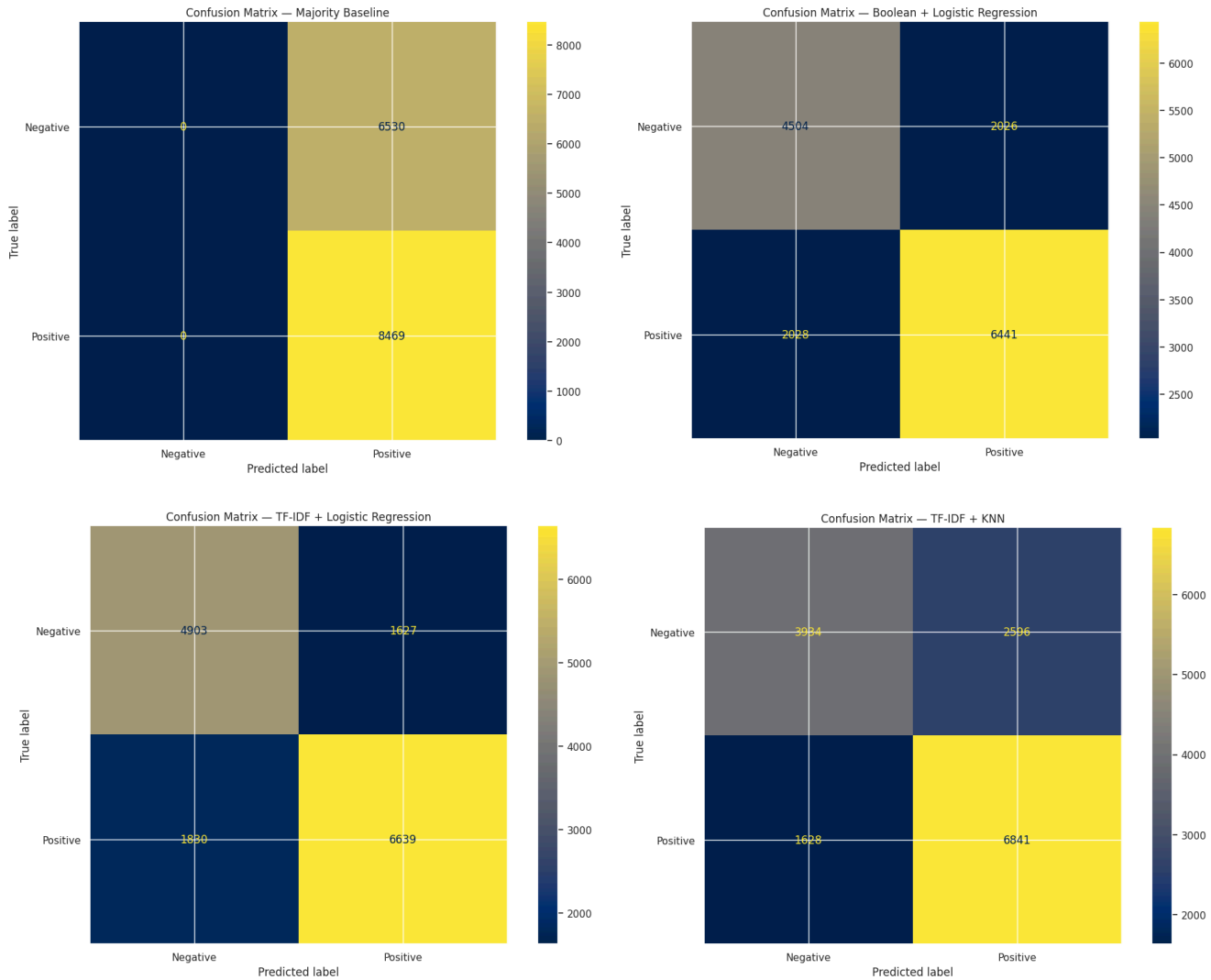
Boolean LR trails by **~0.041** Macro-F1 and **~0.053** PR-AUC, showing the benefit of TF-IDF's term-frequency scaling and n-grams.

k-NN underperforms on the test set despite **nearly-perfect training performance**, indicating **strong overfitting**. In **high-dimensional sparse TF-IDF space**, **cosine similarity** does **not separate classes** effectively; **nearest neighbors** are often **noisy**, especially for **short informal texts**. This leads to notably lower recall for the Negative class (e.g., $R_0 = 0.6025$ vs LR TF-IDF $R_0 = 0.7508$).

Best overall: LR (TF-IDF) with Macro-F1=0.766 and Macro PR-AUC=0.843.

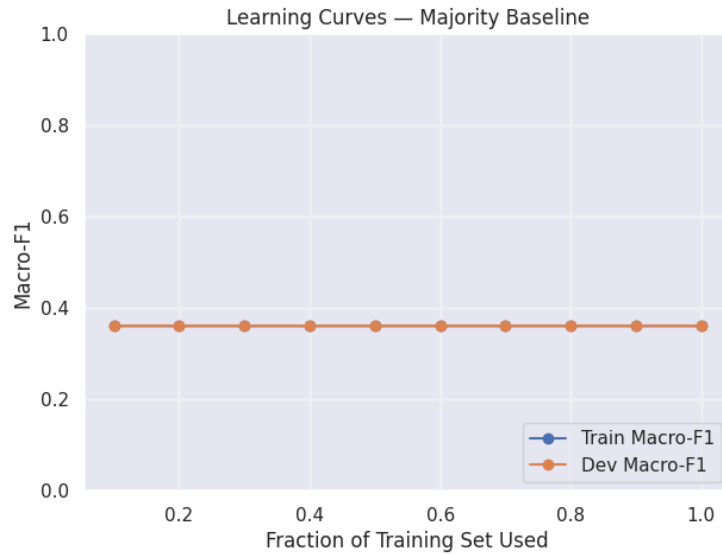
Confusion Matrixes

- **Baseline** assigns **everything to Positive**, yielding **F1(Negative)=0.0** and the diagonal we observed.
- Both **Logistic Regression models (TF-IDF and Boolean)** perform **substantially better**. However, they consistently show lower **recall** on the **Negative** class compared to the **Positive** class. For example, in the **TF-IDF + Logistic Regression** model, we observe $R(\text{Negative})=0.75$ vs. $R(\text{Positive})=0.78$ on the test set. This indicates that the classifier is more likely to miss negative tweets, often predicting them as positive.
- The **KNN** classifier performs **worse** than both **Logistic Regression models**. From the confusion matrix, KNN frequently labels Negative tweets as Positive, resulting in **low recall** for the **Negative class**. This happens because **KNN relies on distance similarity**, which is **ineffective** in **high-dimensional sparse TF-IDF space**, making it **unable to distinguish subtle sentiment cues**. Hence, **KNN is not suitable** for this **task** compared to **Logistic Regression**.

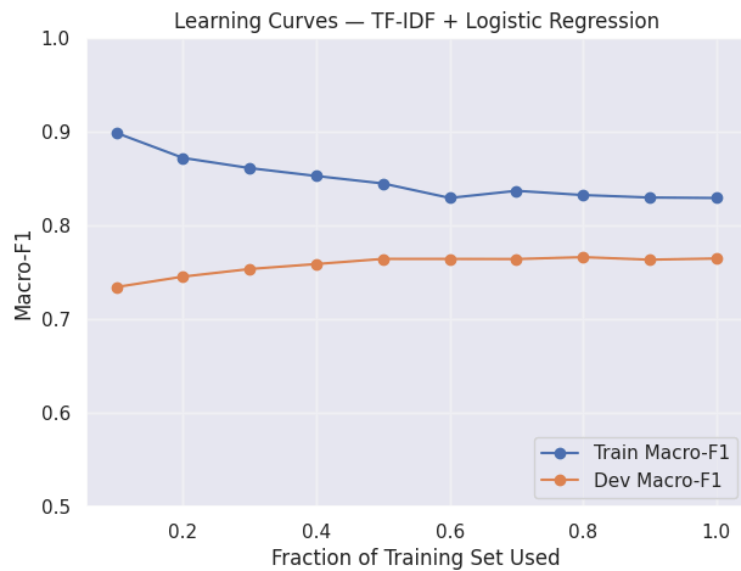


Learning curves (*Train-subset vs Dev; Macro-F1*)

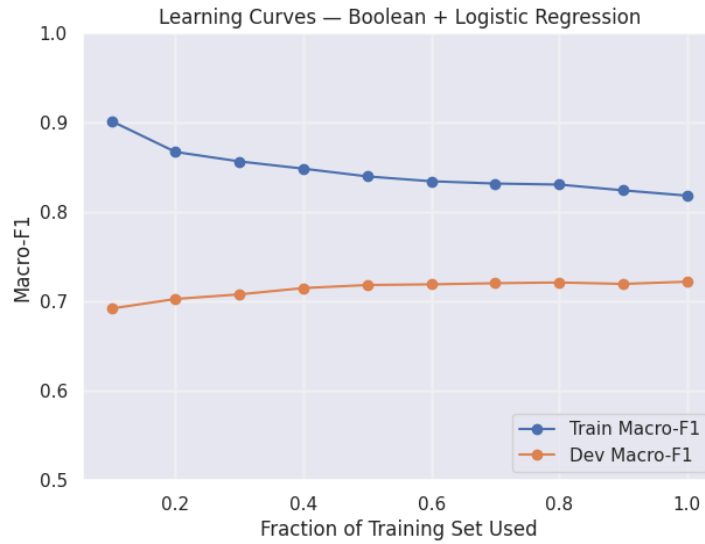
We plot **two curves** per model: **Macro-F1** on (i) the **train subset** (*10%→100%*) and (ii) the **entire Dev set**.



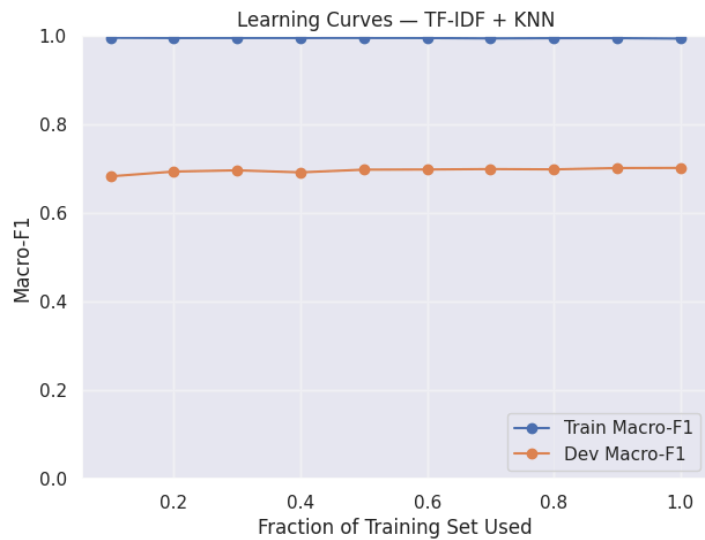
- **Baseline:** Train and Dev curves overlap and are flat at $\approx 0.36 \rightarrow$ the model **does not** learn from more data.



- **LR (Boolean):** Same qualitative trend but consistently **below TF-IDF**, confirming **TF-IDF's advantage** for tweets.



- **LR (*TF-IDF*): Dev Macro-F1 rises steadily from ~0.73 to ~0.76 as data increases; the Train-Dev gap shrinks, indicating reduced overfitting and good generalization.**



- The **k-NN (*TF-IDF*)** model **severely overfits**, achieving **nearly perfect training performance** but **much lower development performance**, showing that it **memorizes training examples rather than learning a generalizable decision boundary**.